

How does an LLM generate text?

Shreya Havaldar

Announcements

- You don't need to email for an absence!!
 - You also don't need to make up in-class labs/activities
- Post on Ed for any questions about HW 1

Today's Lecture

1. **Pre-training**
2. Encoders & decoders
3. Decoding strategies

Tokens are the model's “alphabet”

the model does not generate characters or words directly; it generates **tokens** from a finite vocabulary V .

Tokens can be subwords (“un”, “believ”, “able”), punctuation, whitespace markers, or whole words.

Tokens $\rightarrow$ Logits

Let a tokenizer map text to token IDs:

$$\text{text} \rightarrow x_{1:T}, \quad x_t \in \{1, \dots, |V|\}$$

The LLM outputs **logits** $z \in R^{|V|}$ at each position t .

Logits are unnormalized scores.

Intuition

Think of an LLM as a very large conditional probability table that generalizes across phrases and sentences.

It learns patterns like:

- after “peanut butter and”, “jelly” is likely

- after “import numpy as”, “np” is likely

- after “The capital of France is”, “Paris” is likely.

Logits $\rightarrow$ Probabilities

Using the **softmax** function, we can get a probability distribution across tokens.

$$p(x_{t+1} = v \mid x_{\leq t}) = \frac{\exp(z_t[v])}{\sum_{u \in V} \exp(z_t[u])}$$

Apply softmax to these vectors

$$p(x_{t+1} = v \mid x_{\leq t}) = \frac{\exp(z_t[v])}{\sum_{u \in V} \exp(z_t[u])}$$

< 1, 1, 1, 1, 1 > → ?

< 100, 0, 0, 0, 0 > → ?

< 3, -1, 0, 2, 1 > → ?

Apply softmax to these vectors

$$p(x_{t+1} = v \mid x_{\leq t}) = \frac{\exp(z_t[v])}{\sum_{u \in V} \exp(z_t[u])}$$

$\langle 1, 1, 1, 1, 1 \rangle \rightarrow \langle \mathbf{0.2}, \mathbf{0.2}, \mathbf{0.2}, \mathbf{0.2}, \mathbf{0.2} \rangle$

$\langle 100, 0, 0, 0, 0 \rangle \rightarrow \sim \langle \mathbf{1}, \mathbf{0}, \mathbf{0}, \mathbf{0}, \mathbf{0} \rangle$

$\langle 3, -1, 0, 2, 1 \rangle \rightarrow \langle \mathbf{0.64}, \mathbf{0.01}, \mathbf{0.03}, \mathbf{0.23}, \mathbf{0.09} \rangle$

Logits $\rightarrow$ Probabilities

Logits are “**scores.**” Softmax turns them into a **distribution.**

If the top logit is much larger than the rest, the distribution becomes extremely peaked (essentially deterministic).

If logits are close together, the distribution is flatter (more uncertainty).

Example

Input: "Peanut butter and "

The next-token probabilities could look like:

"jelly": 0.55

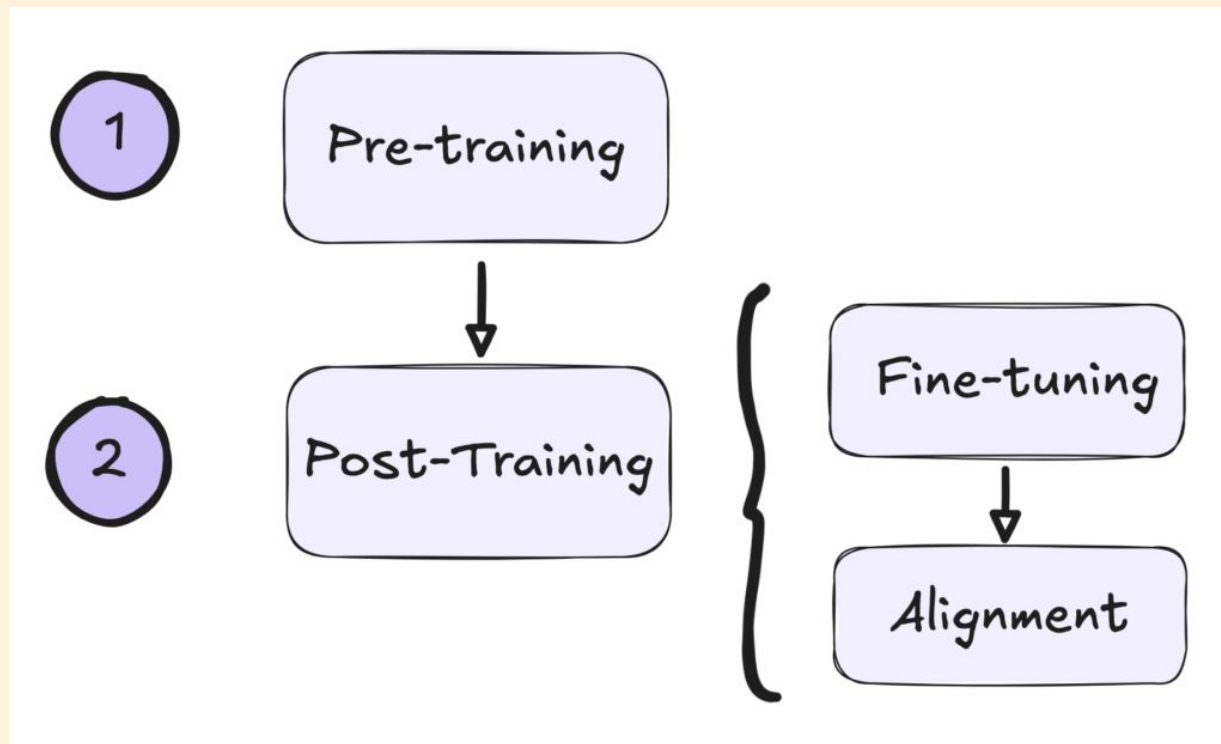
"honey": 0.20

"banana": 0.15

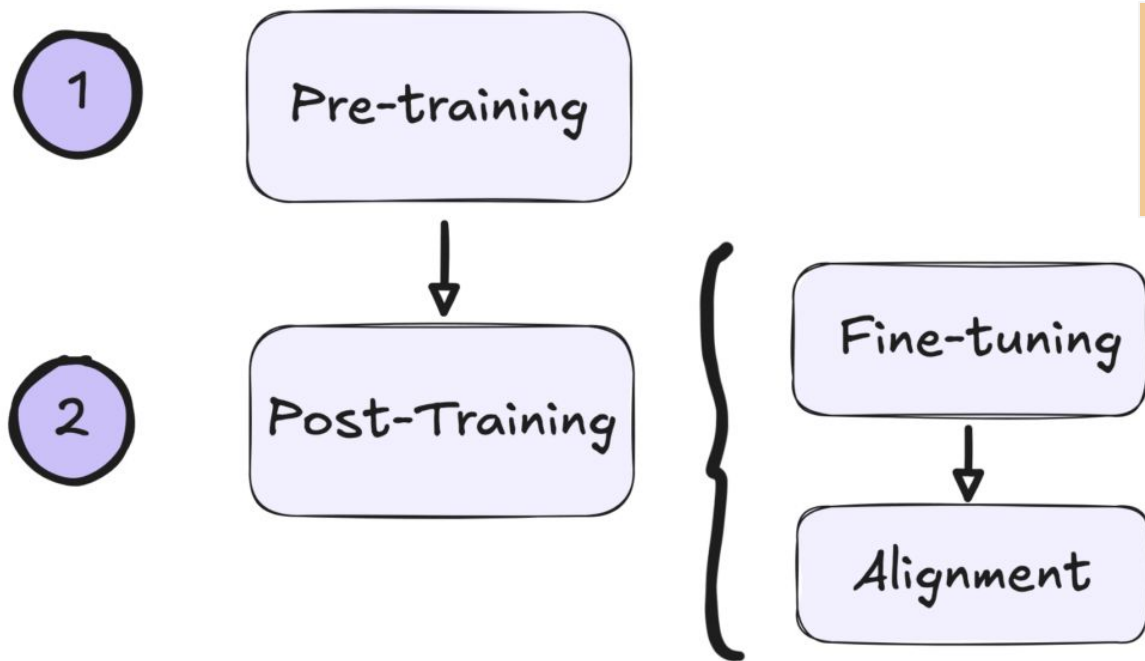
"marshmallow": 0.08

"broccoli": 0.02

2 stages of training an LLM

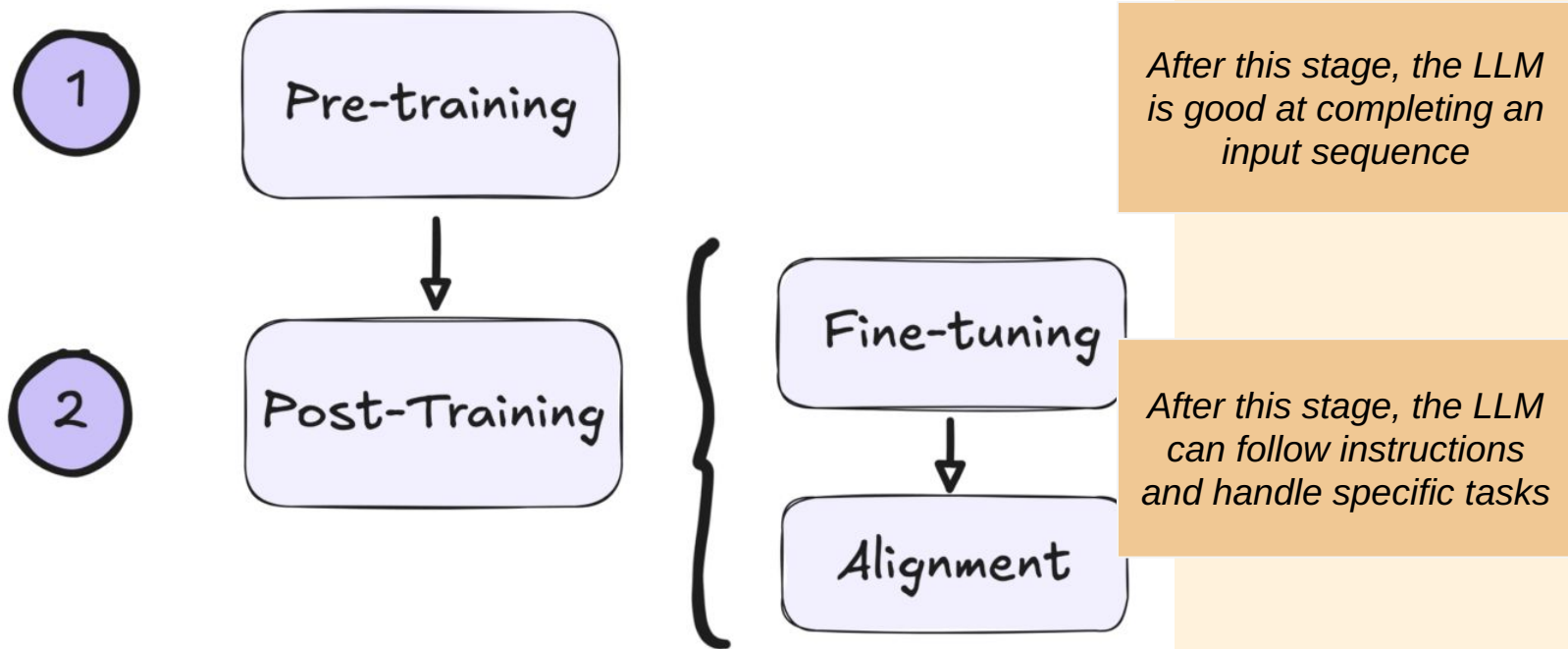


2 stages of training an LLM



After this stage, the LLM is good at completing an input sequence

2 stages of training an LLM



Pre-training

Pre-training builds an LLM's broad world knowledge using next-token prediction

Pre-training is usually **self-supervised**:

- The data is the raw text
- The supervision signal (label) is derived from the text itself
- No need for supervised data: the LLM is trained to predict missing pieces of text

Masked language modeling

Fill-in-the-blank-token prediction (BERT-style)

Objective:

$$\max_{\theta} \sum_{t \in M} \log p_{\theta}(x_t \mid x_{\setminus M})$$

Input: “I made a peanut **<mask>** and jelly sandwich”

Output: “butter”

Masked language modeling

BERT masks some tokens and predicts them using both left and right context.

What the model learns:

- Very strong **bidirectional representations**
 - Each token representation can incorporate information from both sides of the token.
- Great for **understanding tasks** (classification, sentiment analysis) because representations are conditioned on the whole sentence.

Causal language modeling

Next-token prediction (GPT-style)

Objective:
$$\max_{\theta} \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t})$$

Input: I made a peanut butter and jelly **<continuation>**

Output: sandwich

Causal language modeling

GPT only predicts continuations of input phrases/sentences

What the model learns:

- To compress patterns of language into parameters such that it can predict **plausible continuations**.
- It learns syntax (to continue grammatically), semantics (to continue coherently), and a large amount of factual co-occurrence (to continue “correctly” in typical contexts).

Unsupervised training data: Masked vs. Causal

Given this sentence: I made a peanut butter and jelly sandwich

- Make 5 training examples for MLM
- Make 5 training examples for Causal LM

Unsupervised training data: Masked vs. Causal

Given this sentence: I made a peanut butter and jelly sandwich

- Make 5 training examples for **MLM**

1. <MASK> made a peanut butter and jelly sandwich → *I*
2. I <MASK> a peanut butter and jelly sandwich → *made*
3. I made <MASK> peanut butter and jelly sandwich → *a*
4. I made a <MASK> butter and jelly sandwich → *peanut*
5. I made a peanut <MASK> and jelly sandwich → *butter*

* *In practice, many tokens are masked at once (~15%)*

Unsupervised training data: Masked vs. Causal

Given this sentence: I made a peanut butter and jelly sandwich

- Make 5 training examples for **Causal LM**

1. I <MASK> → *made*
2. I made <MASK> → *a*
3. I made a <MASK> → *peanut*
4. I made a peanut <MASK> → *butter*
5. I made a peanut butter <MASK> → *sandwich*

Today's Lecture

1. Pre-training
2. **Encoders & decoders**
3. Decoding strategies

Recap: Transformer

Original transformer (Google, 2017) was trained to do translation:

Called sequence-to-sequence or “seq2seq”

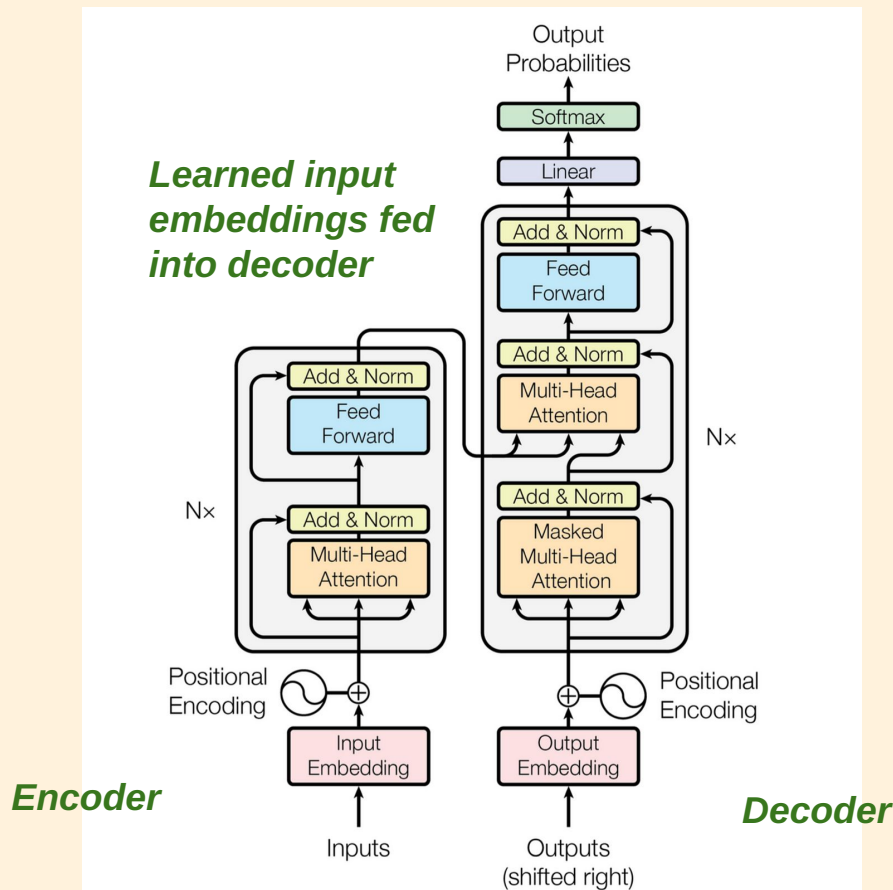


Figure 1: The Transformer - model architecture.

Encoder-only Transformer (BERT family)

Attention pattern: bidirectional self-attention

Pre-training: masked language modeling

Outputs: a vector for each token position (contextual embeddings).

Common use cases:

- Sentiment classification: map a piece of text → positive/negative
- Named entity recognition: label each token as person, location, etc.
- Retrieval embeddings: Embed queries and documents to find matches (used for RAG)

Decoder-only Transformer (GPT family)

Attention pattern: causal self-attention mask

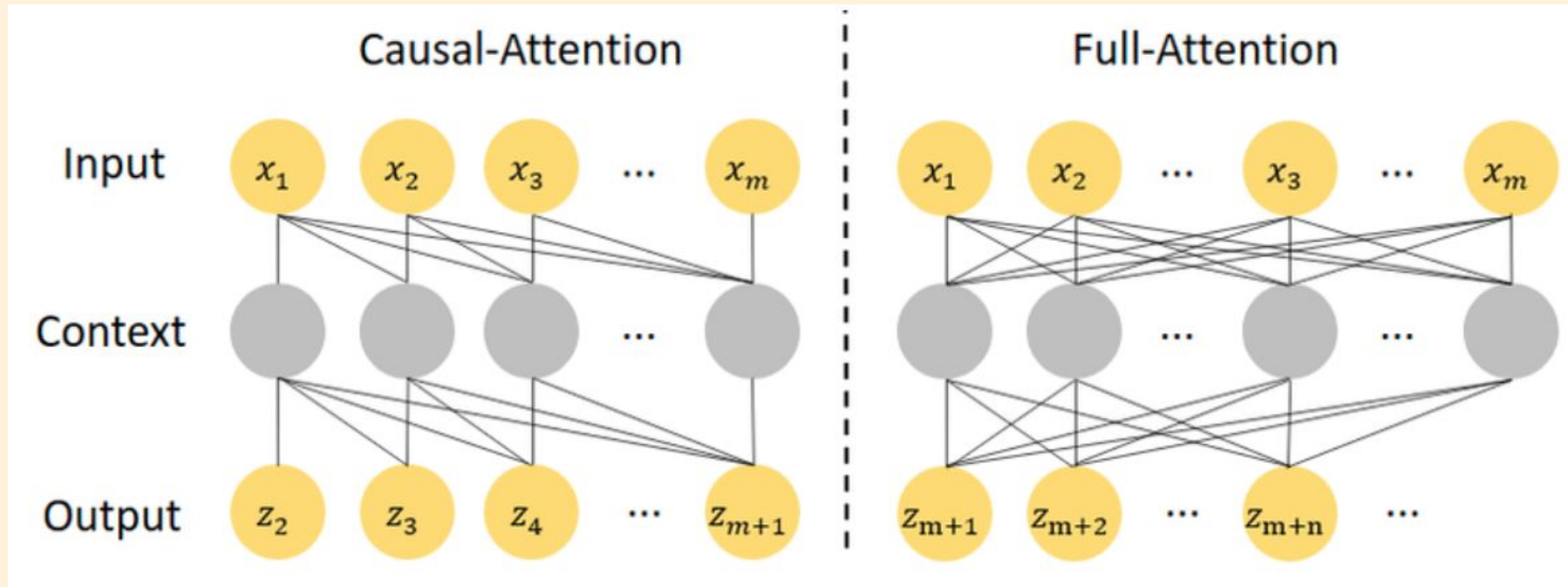
Pre-training: causal language modeling

Outputs: at each position, a distribution for the next token.

Common use cases:

- Chat and instruction following (after post-training)
- Code completion
- Open-ended generation

Causal vs. bidirectional attention



Example: “Your journey starts with one step”

Encoder-only attention

	Your	journey	starts	with	one	step
Your	0.19	0.16	0.16	0.15	0.17	0.15
journey	0.20	0.16	0.16	0.14	0.16	0.14
starts	0.20	0.16	0.16	0.14	0.16	0.14
with	0.18	0.16	0.16	0.15	0.16	0.15
one	0.18	0.16	0.16	0.15	0.16	0.15
step	0.19	0.16	0.16	0.15	0.16	0.15

Attention weight for input tokens
corresponding to “step” and “Your”

Decoder-only attention

	Your	journey	starts	with	one	step
Your	1.0					
journey	0.55	0.44				
starts	0.38	0.30	0.31			
with	0.27	0.24	0.24	0.23		
one	0.21	0.19	0.19	0.18	0.19	
step	0.19	0.16	0.16	0.15	0.16	0.15

Masked out
future tokens
for the “Your”
token

Example: “Your journey starts with one step”

Encoder-only attention

	Your	journey	starts	with	one	step
Your	0.19	0.16	0.16	0.15	0.17	0.15
journey	0.20	0.16	0.16	0.14	0.16	0.14
starts	0.20	0.16	0.16	0.14	0.16	0.14
with	0.18	0.16	0.16	0.15	0.16	0.15
one	0.18	0.16	0.16	0.15	0.16	0.15
step	0.19	0.16	0.16	0.15	0.16	0.15

Attention weight for input tokens corresponding to “step” and “Your”

Decoder-only attention

	Your	journey	starts	with	one	step
Your	1.0					
journey	0.55	0.44				
starts	0.38	0.30	0.31			
with	0.27	0.24	0.24	0.23		
one	0.21	0.19	0.19	0.18	0.19	
step	0.19	0.16	0.16	0.15	0.16	0.15

Masked out
future tokens
for the “Your”
token

*The masked values
are typically set to
negative infinity.
Why?*

Encoder-decoder Transformer (seq2seq)

1. The encoder reads input bidirectionally
2. The decoder generates output autoregressively (one token at a time)
3. **Decoder cross-attention:** decoder attends to encoder's representations to generate output conditioned on the source sequence.

Most common use case is Translation:

Input sentence in French →

Output sentence in English

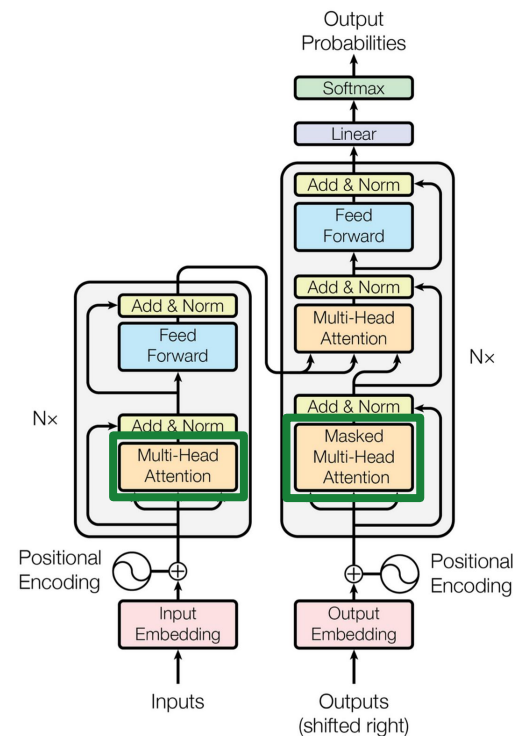


Figure 1: The Transformer - model architecture.

Discussion (10 min)

1. How does the causal attention mask in a decoder-only Transformer prevent “cheating” during training?
2. Suppose you want to build a system to classify the number of stars on a given Yelp review. What architecture would you start with? What would you add? How would you train it?
3. If you tried to use an MLM-pretrained encoder-only model to generate a paragraph left-to-right, what workaround could you use (conceptually) to still generate text?

Today's Lecture

1. Pre-training
2. Encoders & decoders
3. **Decoding strategies**

The generation loop

During generation, an LLM:

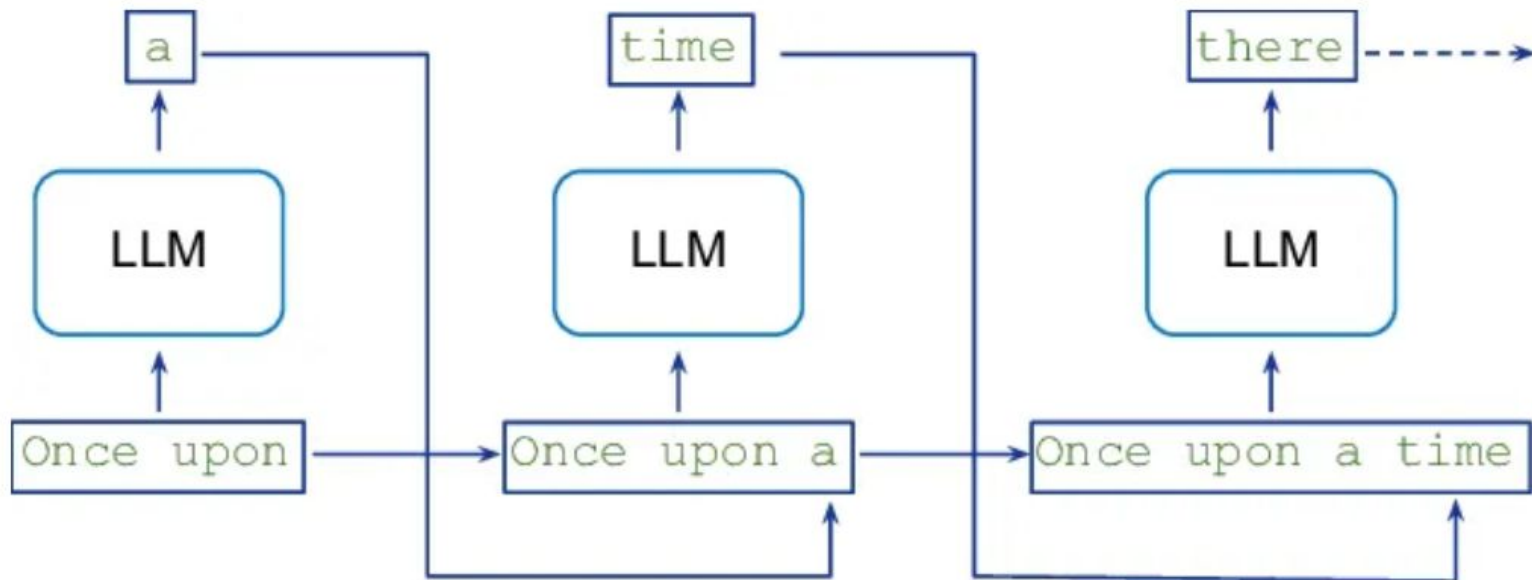
1. Tokenizes the input
2. Runs a forward pass
3. Gets a set of logits for every token in its vocabulary
4. Converts these logits to a probability distribution over tokens
5. Selects the output token using this distribution

Autoregressive decoding

Given prompt tokens $x_{1:t}$

1. Compute logits for next token.
2. Optionally transform logits (temperature, repetition penalty, filters).
3. Choose next token using a decoding strategy.
4. Append token to context.
5. Repeat until stop condition.

Autoregressive decoding



Autoregressive Sampling

Chat Generation

Generation models are trained using causal language modeling:

- At inference, you give a prefix (your prompt) and ask for a continuation.
- The objective is aligned with the usage.

MLM models (like BERT) are not the default for chat generation:

- MLM does not define “generate a continuation” directly.
- You can generate by iteratively masking and filling, but it is less natural than autoregressive decoding.

Temperature

Temperature controls randomness

Modify logits: $z'[v] = z[v] / T$

$T < 1$: sharper distribution $\rightarrow$ more deterministic

$T > 1$: flatter distribution $\rightarrow$ more diverse/random

- If the model is very confident (top token ~ 0.90), changing temperature might not matter much.
- If it is uncertain (top token ~ 0.20), temperature can drastically alter diversity.

Example: Temperature (10 min)

Input: Once upon a time, there was a

LLM output logits:

princess: 5.2

mushroom: 0.9

monster: -2.3

frog: 3.2

spider: 1.3

$$P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Using the above softmax with temperature equation, calculate the output distribution for:

$$T = 1.0$$

$$T = 0.0001$$

$$T = 50$$

Example: Temperature (10 min)

Input: Once upon a time, there was a

Token	Logit	T = 1.0	T = 0.0001	T = 50
princess	5.2			
mushroom	0.9			
monster	-2.3			
frog	3.2			
spider	1.3			

Example: Temperature (10 min)

Input: Once upon a time, there was a

Token	Logit	T = 1.0	T = 0.0001	T = 50
princess	5.2	0.8549		
mushroom	0.9	0.0116		
monster	-2.3	0.0005		
frog	3.2	0.1157		
spider	1.3	0.0173		

Example: Temperature (10 min)

Input: Once upon a time, there was a

Token	Logit	T = 1.0	T = 0.0001	T = 50
princess	5.2	0.8549	1.0000	
mushroom	0.9	0.0116	0.0000	
monster	-2.3	0.0005	0.0000	
frog	3.2	0.1157	0.0000	
spider	1.3	0.0173	0.0000	

Example: Temperature (10 min)

Input: Once upon a time, there was a

Token	Logit	T = 1.0	T = 0.0001	T = 50
princess	5.2	0.8549	1.0000	0.2144
mushroom	0.9	0.0116	0.0000	0.1967
monster	-2.3	0.0005	0.0000	0.1845
frog	3.2	0.1157	0.0000	0.2060
spider	1.3	0.0173	0.0000	0.1983

Greedy decoding

Rule: choose highest probability token at every step.

$$\hat{x}_{t+1} = \arg \max_v p(v \mid x_{\leq t})$$

Strength: stable, reproducible.

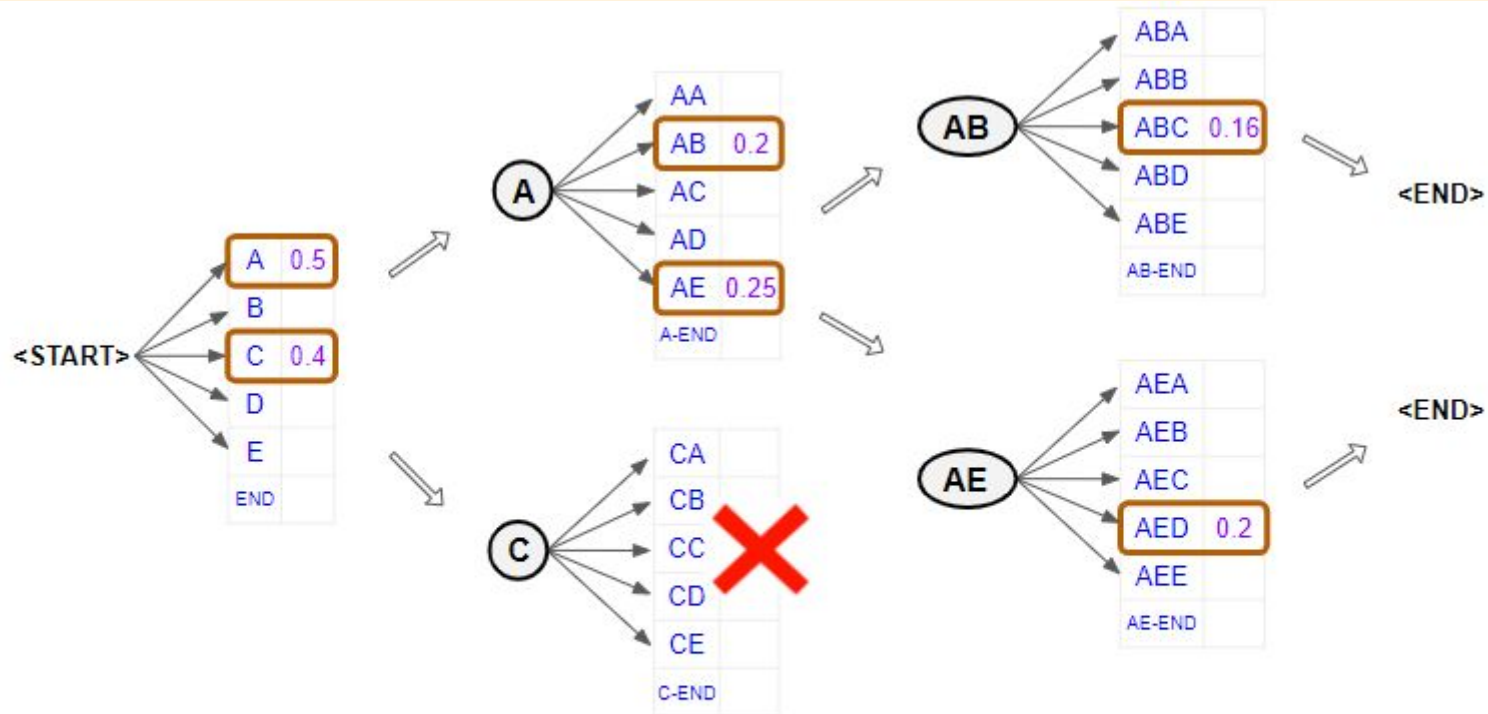
Weakness: can fall into locally optimal continuations that are globally poor.
Produces generic tropes + repetitive phrasing because these are high-probability continuations across many training examples.

Beam search

Rule: Maintain top B candidate partial sequences ranked by cumulative log probability

$$\text{score}(x_{1:t}) = \sum_{i=1}^t \log p(x_i \mid x_{<i})$$

** log-probabilities used to prevent numerical underflow + convert computationally expensive multiplication into efficient addition*



Candidate
Sequences

A, C

Position 1

AB, AE

Position 2

ABC, AED

Position 3

Beam search

Weakness: Not ideal for chat applications

- Beam search optimizes likelihood, which tends to favor generic, high-frequency responses (“As an AI language model...”) and reduces diversity.

Top-k sampling

Rule: Restrict to k highest-prob tokens, renormalize, sample.

Strength: Ensures the LLM does not sample from the “weird tail” of the distribution

Weakness: fixed k is crude; sometimes you want a small set (when confident), sometimes a larger set (when uncertain).

Nucleus sampling (top-p)

Rule: Choose the smallest set whose cumulative probability $\geq p$, then sample.

$$\sum_{v \in P} p(v \mid x_{\leq t}) \geq p$$

Strength: Adapts to model uncertainty.

Repetition and frequency penalties

Many APIs add heuristics that reduce probability of tokens that have already appeared frequently in the generated text.

- This limits loops of the same text during generation

Standard “recipe”

- Temperature ~ 0.7 – 1.0
 - Higher temperature = more diverse/creative generations
- top-p ~ 0.9 – 0.95

Activity (Rest of class)

Complete ***beam_search.py*** (on the class website)

Quiz Code: **3BP5V8**